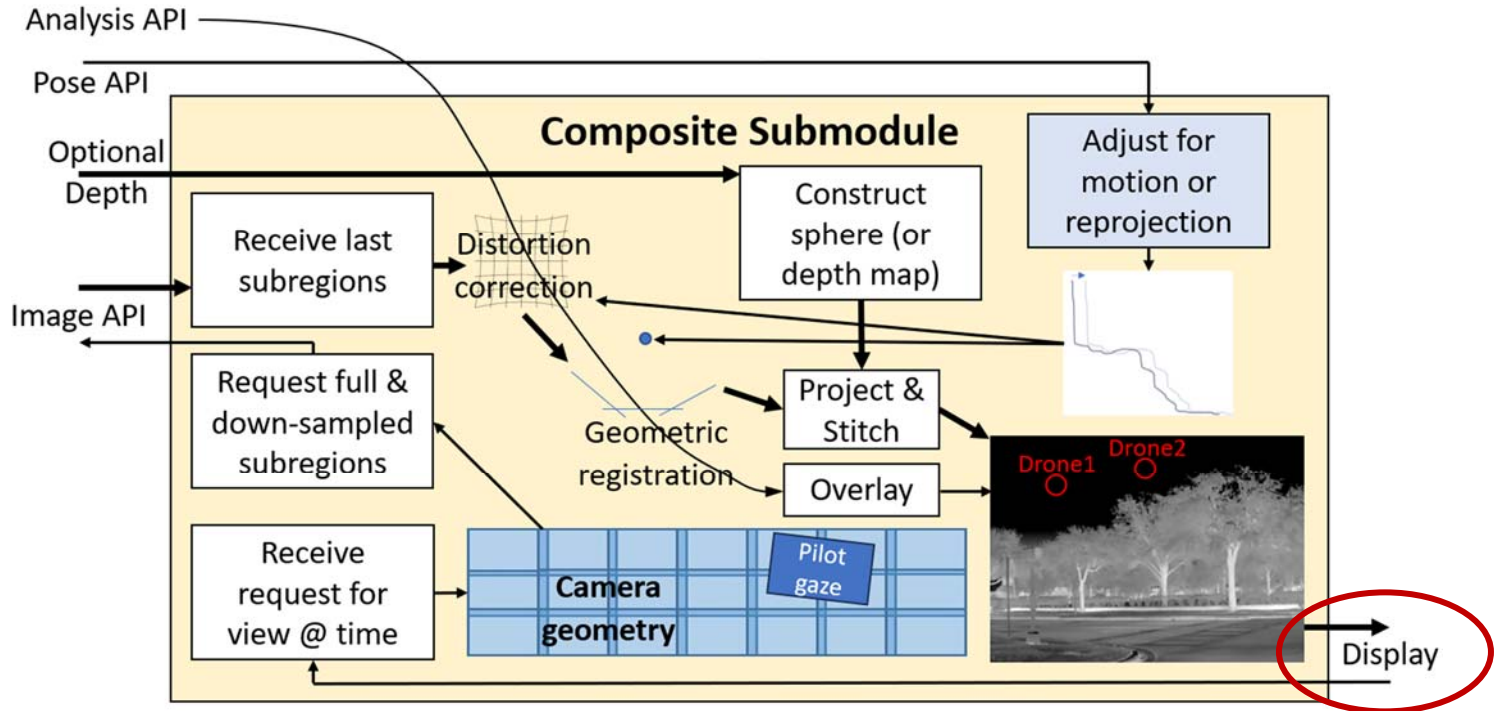


# Interfacing With Render Module

This document describes the interfaces available for displays and the *ASDP\_Render\_Module* program.

## Overview



The figure above shows the conceptual interface between physical display hardware (lower right) and the *Composite* submodule of the *ASDP\_Render\_Module* program along with the conceptual operations that the program performs to produce the images. It receives view requests and produces displayed frames. Depending on the display type, these may be internal GPU buffers or external video feeds.

The submodule renders a set of camera frames into a consistent helicopter-based space, warped to remove camera distortions and moved to match their predicted location at the requested render time. Both linear (for HMD) and cylindrical (for large-screen wide-field-of-view displays) projections are supported to produce a rendered view with the specified pose. Iconography indicating targets are overlaid when optional analysis modules are connected.

For camera balls equipped with stereo depth cameras, the camera images are projected onto a manifold with different distances in different directions. For non-stereo system, the display depth is fixed across the scene, defaulting to 900m.

The system supports multiple displays at the same time. Elbit XSight + 8K flat-screen display and OpenXR (Varjo XR-4) + flat-screen HD display are the two configurations most often used for demonstrations.

## Inputs from display

The Composite submodule receives input from the physical display device that describes the following:

- the **viewport(s)** to be rendered, including pose (position and orientation) and pose velocity,
- the **time** relative to a CPU high-resolution timer at which the image will be presented to the viewer, and
- (optional) triggers for various operations within the system.

The method of providing this information differs based on the type of display. There is a C++ class derived from a base class to implement each type of display and this can be extended to new types of display. The current implementations include the following:

- **DisplayXSight:** (Using specific previous-generation demonstration unit tuned for our system.)
  - **Viewport:** The display resolution and horizontal field of view are specified at construction time and remain fixed throughout the run; the default values are 2560x2048@50fps, hFOV 70. The azimuth, elevation, roll, and an XSight timecode<sup>1</sup> are received from multicast UDP packets.
  - **Time:** The time at which the next image will be scanned to the display, based on the graphics card refresh time on the render server (used to render the scene).
- **DisplayOpenXR:** (Tested using Varjo XR-4; the implementation supports any OpenGL-compatible runtime.)
  - **Viewports:** The two viewpoints (one for each eye in a stereo view) are obtained using *xrLocateViews()*. These are oversized views that will be warped by OpenXR during the rendering process. OpenXR also handles HMD distortion correction and intra-frame warping caused by motion.
  - **Time:** The predicted rendering time is obtained using *xrWaitFrame()*. This produces a value that can be converted to a Windows performance counter and used to determine how far into the future to predict the world state. The same world projection adjusted to the requested time is used for both views.
  - **Triggers:** Buttons on the OpenXR controller can request system quit or other behaviors.
- **DisplayWindow:** (Supports up to 8Kp60 display, full-screen borderless used to reduce display latency.)
  - **Viewport:** The viewport is driven by either a joystick/gamepad or keyboard commands.
  - **Time:** The time at which the next image will be presented to the viewer is based on the graphics card refresh time on the render server and used to render the scene.
  - **Triggers:** Joystick button presses and key presses control various system behaviors: play/pause during replay from storage, IR camera offsets to align brightness and/or gain, etc..

---

<sup>1</sup> This time code is not the time at which to render the frame. It is an opaque XSight time value that is embedded into the rendered image to send back along with the pose information. The XSight hardware uses this to apply additional motion compensation.

## Outputs To Display

The desktide computer can drive HDMI and DisplayPort connectors; the laptop has HDMI and Thunderbolt connectors. A derived class could read back each image and send via 100GigE or other interfaces, but this would introduce latency.

For minimum display latency, full-screen borderless windows should be used to feed data to the rendering hardware. This removes extra operating-system composition steps that cause one or more buffered images in the hardware rendering pipeline. The display devices that are currently implemented include the following:

- **DisplayXSight:** This uses an HDMI cable with a borderless full-screen window to reduce latency. Linear projection is used to produce a rendered view with the specified pose. It sends a custom interleaved monochrome format encoded in color pixels. It also encodes viewpoint and timing information in the upper-left corner of the image (sending the request information back to the XSight to enable motion correction.) The XSight hardware handles motion compensation beyond the display scan-out time, distortion mapping, and intra-frame warping based on the timecode it sent and rendering pose (both of which are embedded in the displayed image).
- **DisplayOpenXR:** The two oversized eye frames are rendered into OpenXR-provided swapchain buffers in GPU memory. Linear projection is used to produce rendered views with the specified poses. OpenXR handles feeding these to the proper display port for the device after warping them for motion compensation.
- **DisplayWindow:** This class uses GLFW to render into either a window or full-screen borderless display. Linear or cylindrical projection is used to produce an image with requested FOVs and resolution.

## Command-line options

ASDP\_Render\_Module has many command-line arguments that control its behavior. The ones relevant to display are as follows:

- **--replay:** Camera flights can be recorded to a Storage Manager. This command requests a replay rather than live.
- **--addDisplay:** More than one display can be supported. This is most often used during demonstrations to enable those not wearing a head-mounted display to see what the viewer is looking at. Each display (including the first) can have its properties set using one or more of the following arguments.
- **--xSight:** Use an Elbit XSight display with the specified network interface and physical display.
- **--openXR:** Use an OpenXR compatible display (Varjo XR-4).
- **--viewpointOffset:** Offset the viewpoint from the center of the camera. This is particularly useful with depth cameras and a head-mounted display because it can shift the center of the view to the pilot's head location rather than the camera location, removing the offset between their see-through view of the world and the camera view of the world and improving their perception of distance to the ground.
- **--width:** Width of the display in pixels.
- **--height:** Height of the display in pixels.
- **--hFOV:** The horizontal field of view for the display (vertical computed based on the height/width).
- **--fullscreen:** Run the display in full-screen mode (borderless) to reduce latency and hide GUI elements.
- **--fps:** Frames/second to run the display at (when full-screen).
- **--enableCP:** This enables cylindrical projection, useful for 8K wide-format displays with a large field of view.